Name – Earl Pratik Sinha

Roll – 2024AD05449

Github - https://github.com/earlpratiksinha/heart-disease-mlops

Video link - https://drive.google.com/file/d/1H8ercOb-Ap6BFAoywdTbK4sRwrROROdS/view?usp=sharing

---

1. Setup & Installation Instructions

This repository contains an end-to-end MLOps pipeline for predicting heart disease using a FastAPI backend, Docker containerization, and a Kubernetes deployment.

Prerequisites

Ensure the following tools are installed on your system before proceeding:

- **Git** (for version control)
- **Python 3.10+**
- **Docker Desktop** (running)
- **Minikube & Kubectl** (for local Kubernetes deployment)

Option A: Local Setup (Development)

1. **Clone the repository:**

Bash

```
git clone https://github.com/earlpratiksinha/heart-disease-mlops.git

cd heart-disease-mlops
```

2. **Create and activate a virtual environment:**

Bash

```
python -m venv venv

# On Windows:

.\venv\Scripts\activate

# On macOS/Linux:

source venv/bin/activate
```

3. **Install dependencies:**

Bash

```
pip install -r requirements.txt
```

4. **Fetch data and train the model:**

Bash

python src/download_data.py

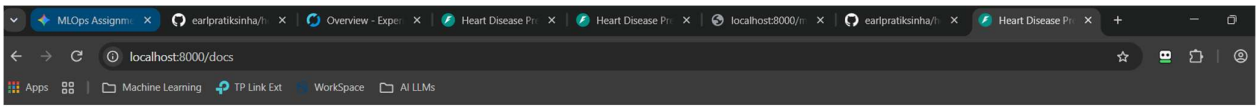
python src/train.py

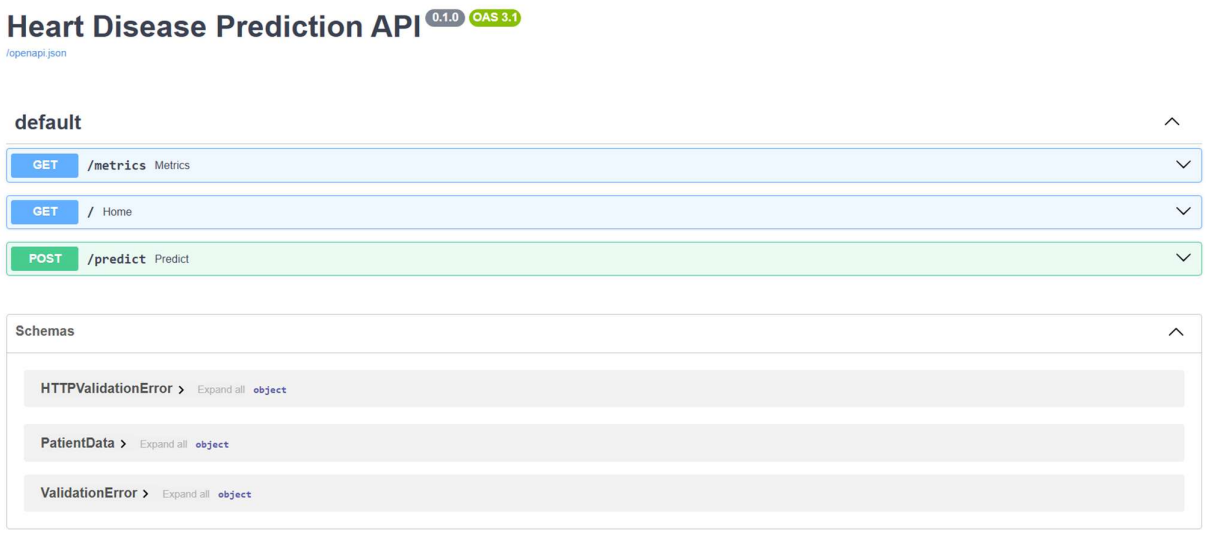
5. **Run the API locally:**

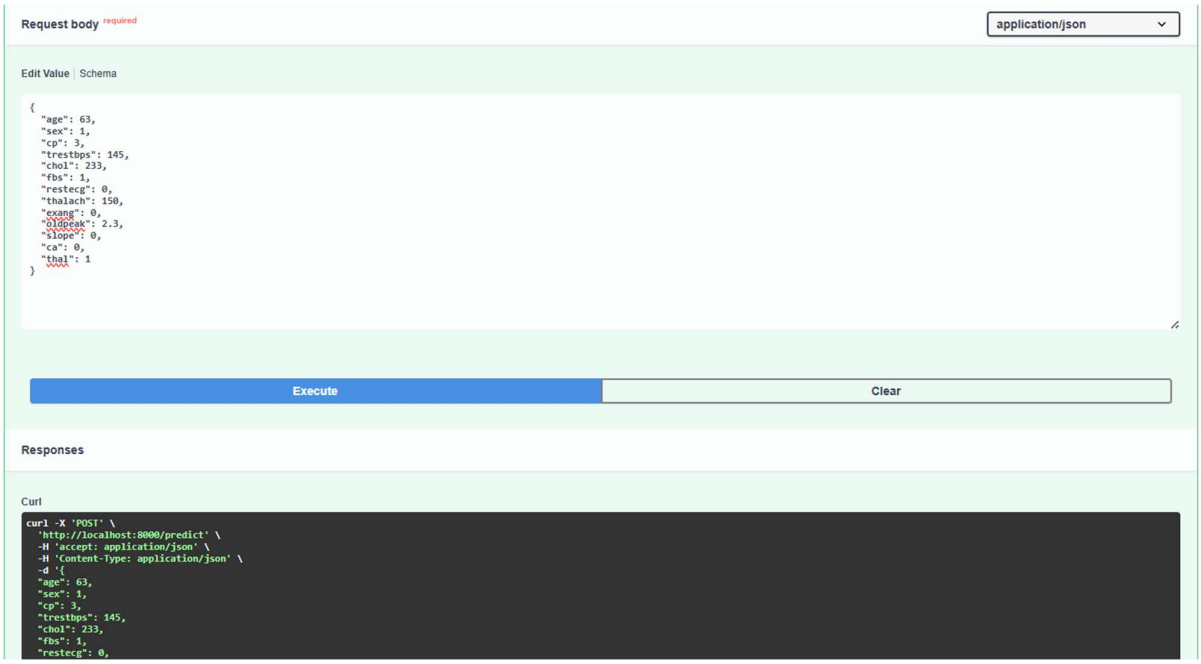
Bash

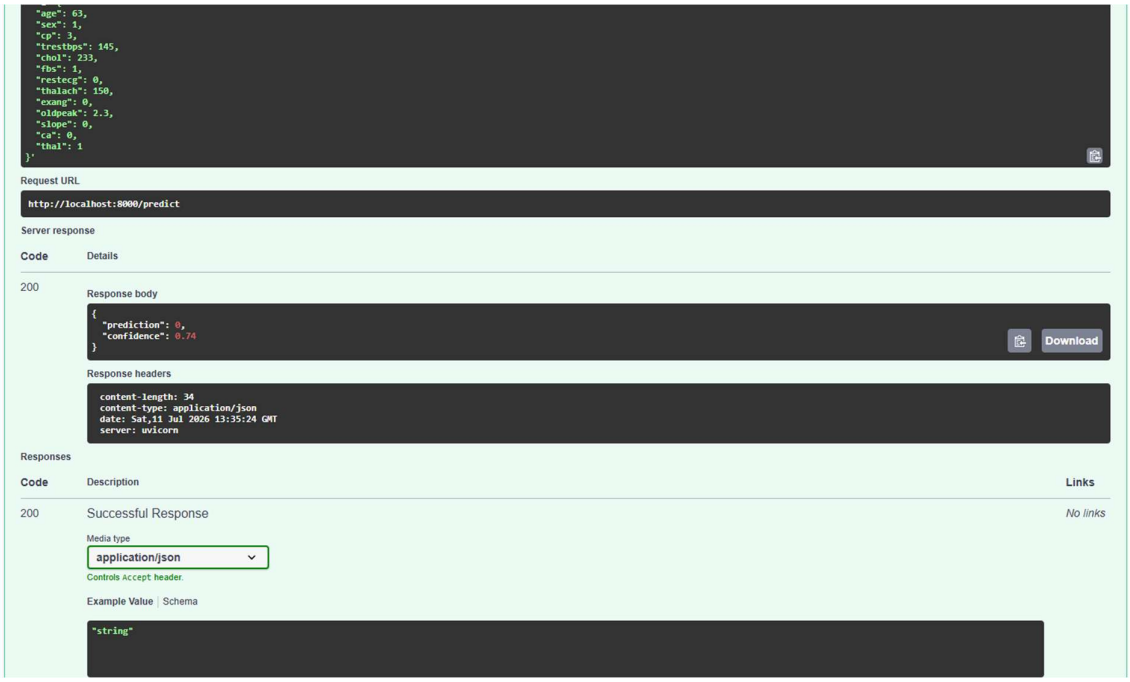
uvicorn src.api:app --reload

The API will be available at http://localhost:8000/docs.







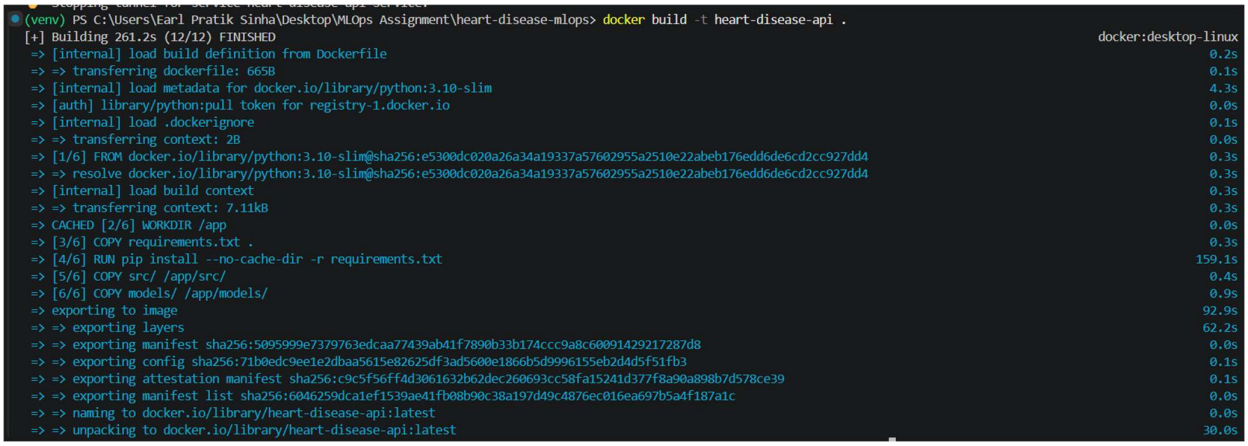


Option B: Docker Setup (Containerized)

1. **Build the Docker image:**

Bash

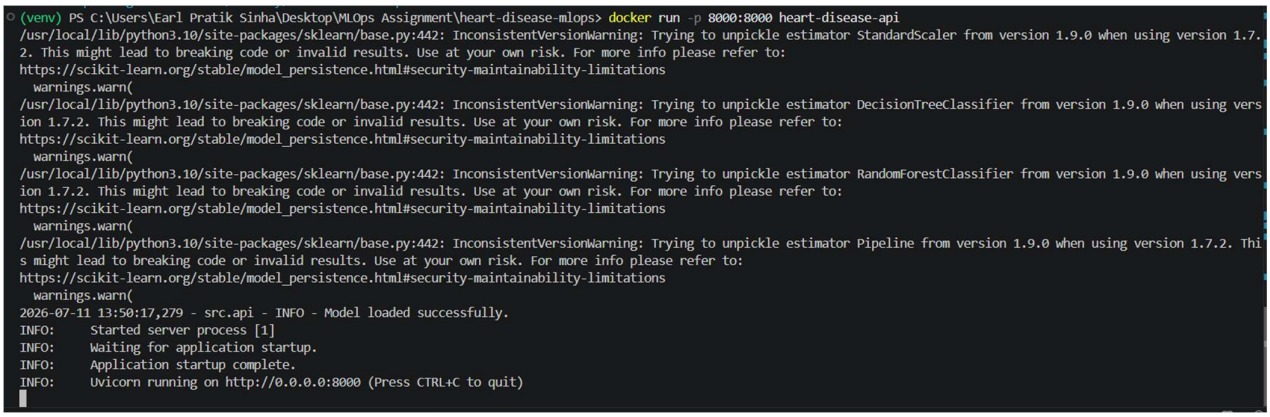
docker build -t heart-disease-api .

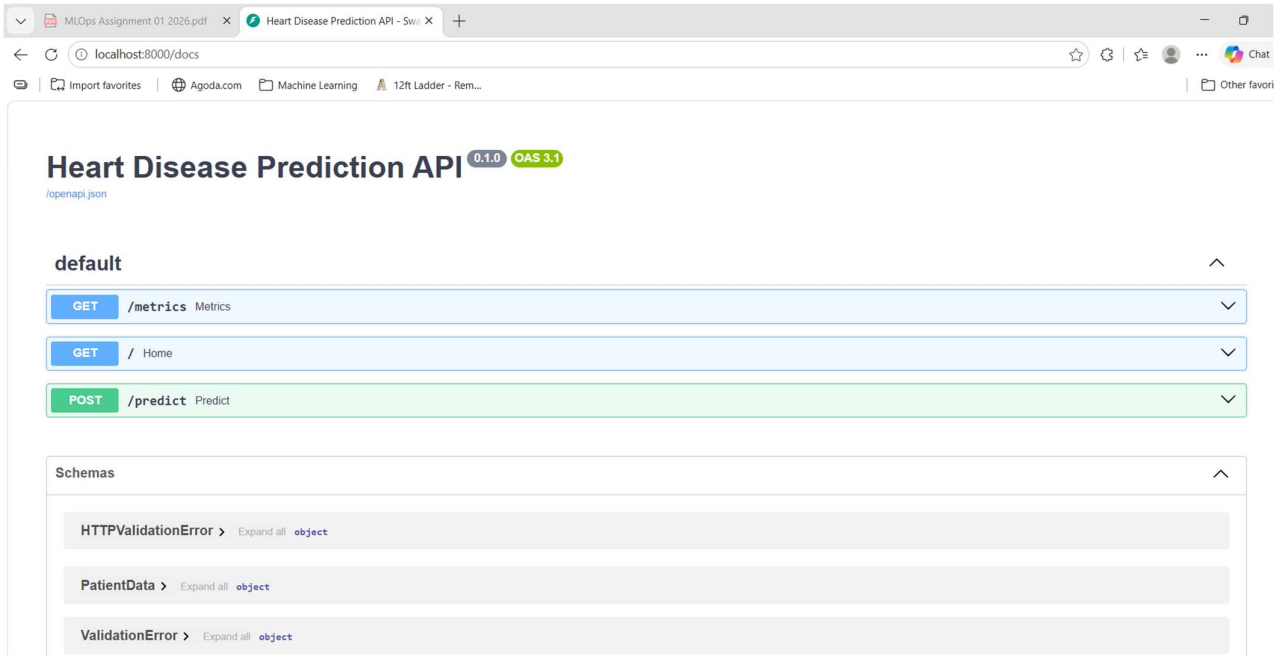


2. **Run the container:**

Bash

docker run -p 8000:8000 heart-disease-api





Option C: Kubernetes Setup (Production/Minikube)

1. **Start Minikube:**

Bash

minikube start

2. **Load the Docker image into Minikube's local registry:**

Bash

minikube image load heart-disease-api:latest

3. **Deploy the application and service:**

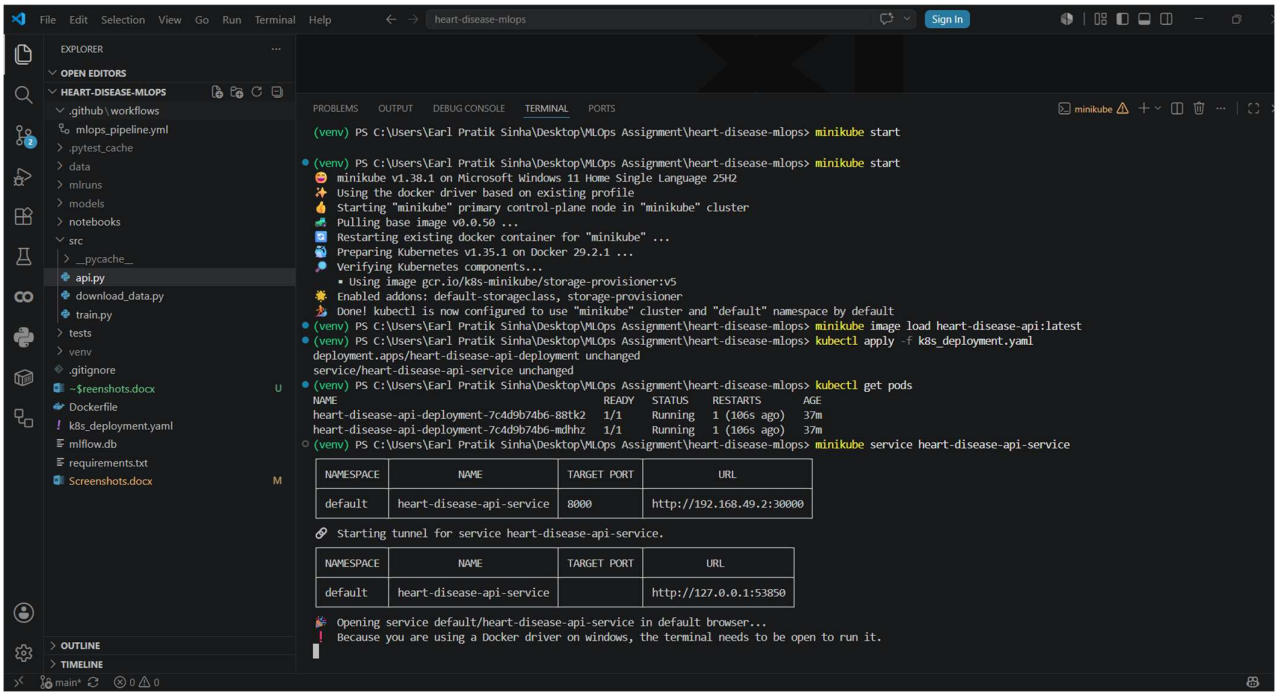
Bash

kubectl apply -f k8s_deployment.yaml

4. **Verify the pods are running:**

Bash

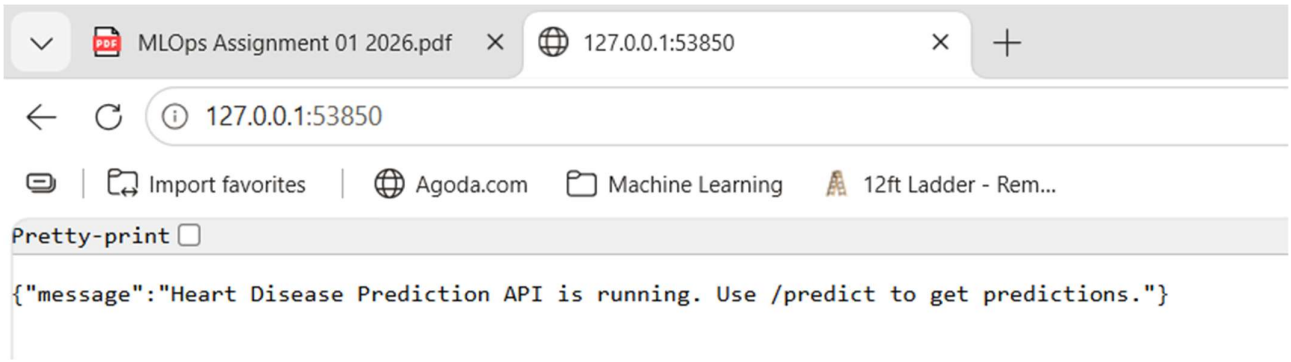
kubectl get pods

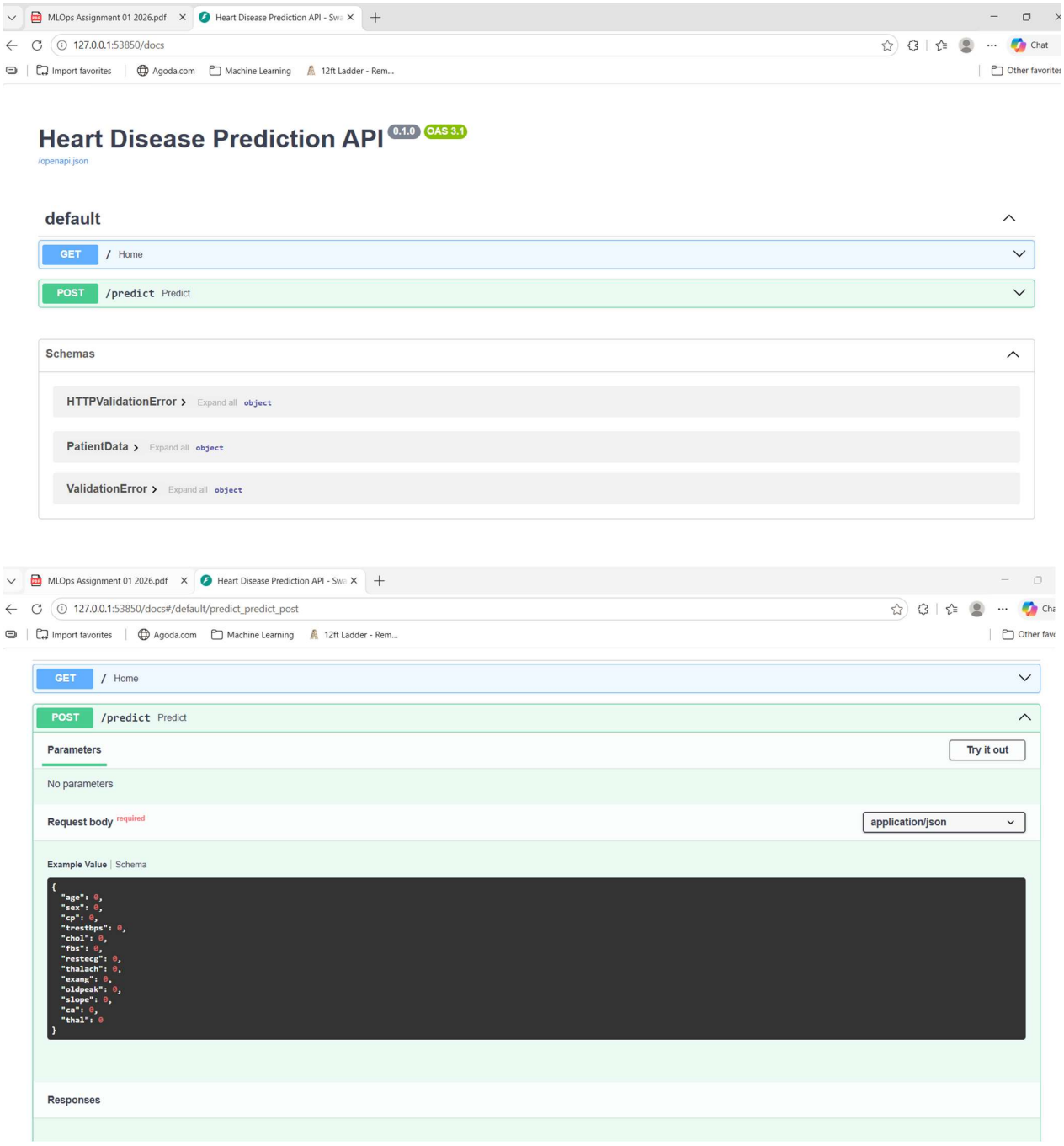


5. **Access the API:**

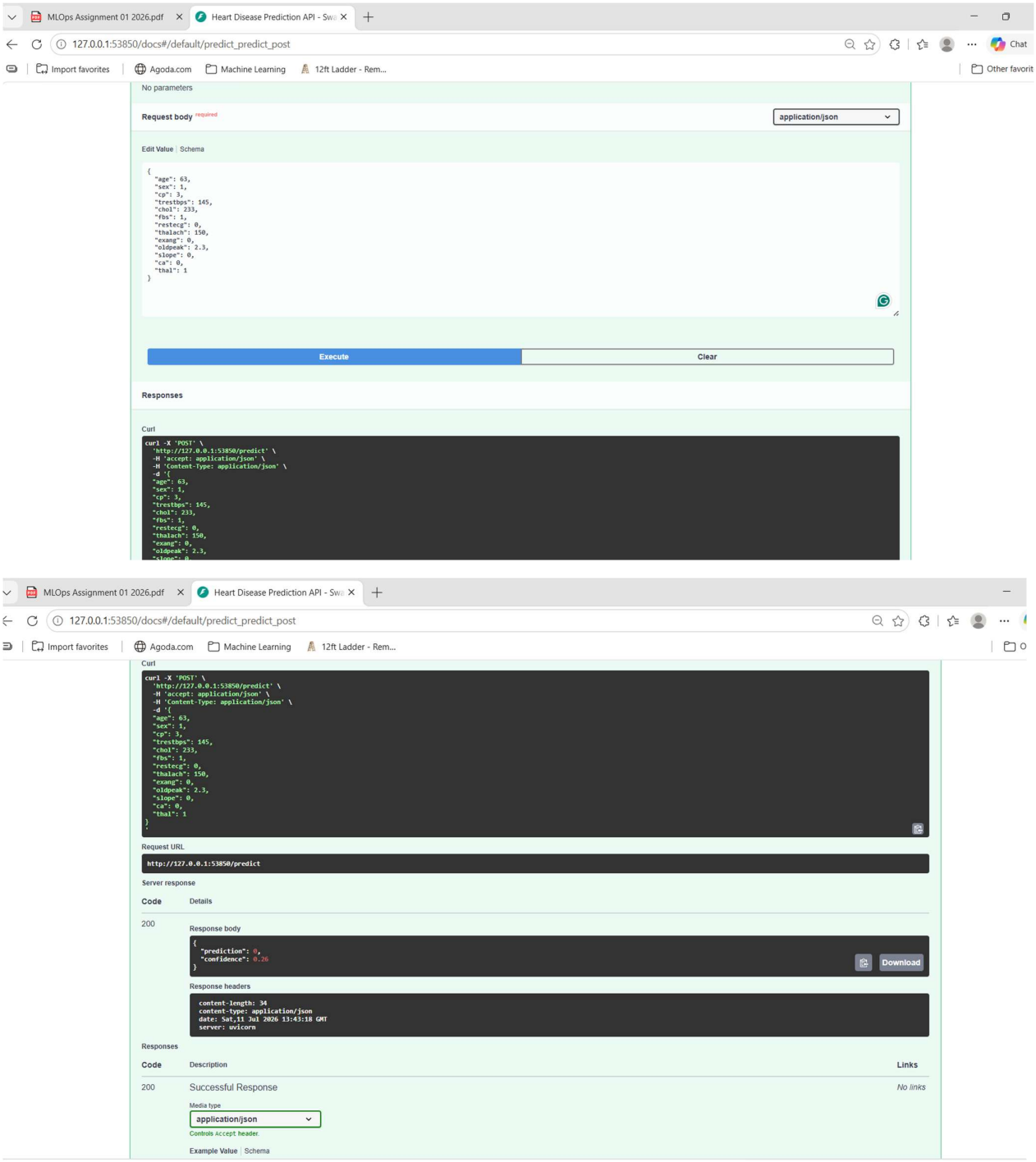
Bash

minikube service heart-disease-api-service




MLOps Assignment 01 2026.pdf
Heart Disease Prediction API - Swa
127.0.0.1:53850/docs
Import favorites
Agoda.com
Machine Learning
12ft Ladder - Rem...
Other favorites
Chat
Heart Disease Prediction API
0.1.0
OAS 3.1
/openapi.json
default
GET / Home
POST /predict Predict
Schemas
HTTPValidationError Expand all object
PatientData Expand all object
ValidationError Expand all object

MLOps Assignment 01 2026.pdf
Heart Disease Prediction API - Swa
127.0.0.1:53850/docs#/default/predict_predict_post
Import favorites
Agoda.com
Machine Learning
12ft Ladder - Rem...
GET / Home
POST /predict Predict
Parameters
Try it out
No parameters
Request body required
application/json
Example Value | Schema
{
"age": 0,
"sex": 0,
"cp": 0,
"trestbps": 0,
"chol": 0,
"fbs": 0,
"restecg": 0,
"thalach": 0,
"exang": 0,
"oldpeak": 0,
"slope": 0,
"ca": 0,
"thal": 0
}
Responses



This command will automatically tunnel and open the API in your default web browser.

2. System Architecture & MLOps Pipeline

This project implements a fully automated end-to-end machine learning lifecycle:

- **Data Ingestion & Processing:** The download_data.py script automatically fetches the Heart Disease dataset from the UCI Machine Learning Repository, engineers the target variable, handles missing values via median imputation, and outputs a clean CSV.

- **Model Training:** The train.py script splits the data, trains a machine learning model, evaluates its accuracy, and serializes the best-performing model using joblib.
- **API Development:** A robust REST API built with **FastAPI** serves the model. It includes data validation via Pydantic and returns both the prediction and the model's confidence score.
- **Containerization:** The application is packaged into an isolated, reproducible environment using **Docker**, ensuring consistency across development and production.
- **CI/CD Automation:** A **GitHub Actions** workflow is configured to trigger on every push to the main branch. It automatically sets up the Python environment, runs the data pipeline, trains the model, executes pytest unit tests, and builds the Docker image.
- **Production Deployment:** The containerized application is deployed to a local **Kubernetes** cluster (Minikube). The deployment configuration (k8s_deployment.yaml) manages a replica set of 2 pods for high availability and exposes the application via a NodePort service.
- **Observability:** The API integrates prometheus-fastapi-instrumentator and Python's logging module to track HTTP request metrics, latency, and prediction events at the /metrics endpoint.

3. API Documentation

POST /predict

Accepts patient health metrics and returns a heart disease prediction.

- **Content-Type:** application/json
- **Example Payload:**

JSON


```
{
  "age": 63,
  "sex": 1,
  "cp": 3,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
```

"thal": 1

}

- **Example Response:**

JSON

{

"prediction": 1,

"confidence": 0.85

}

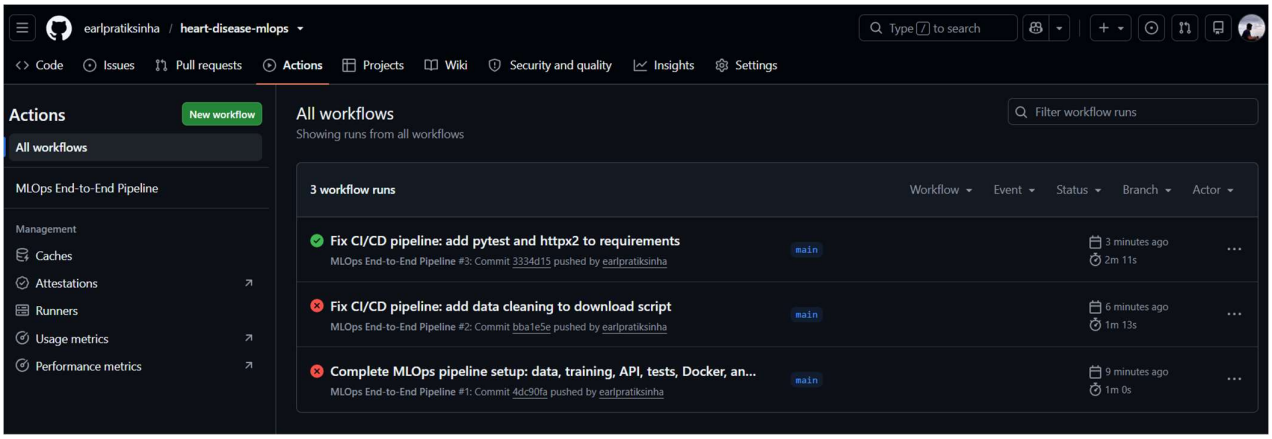
GET /metrics

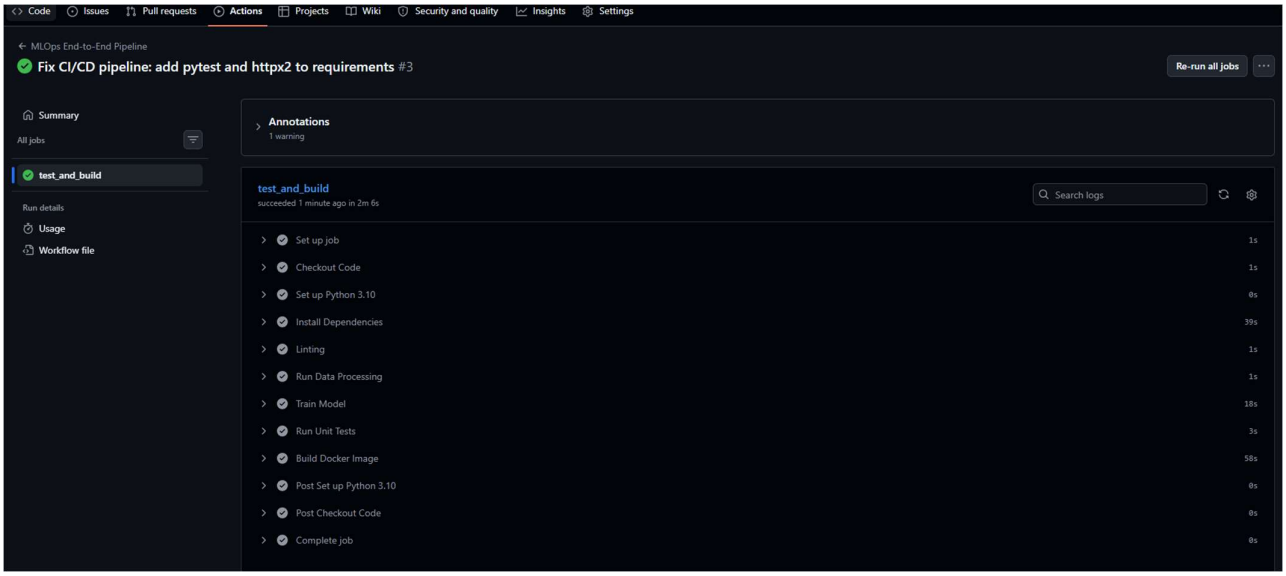
Exposes system and API metrics (requests, latency, memory usage) in plain text format for Prometheus scraping.

4. Proof of Execution

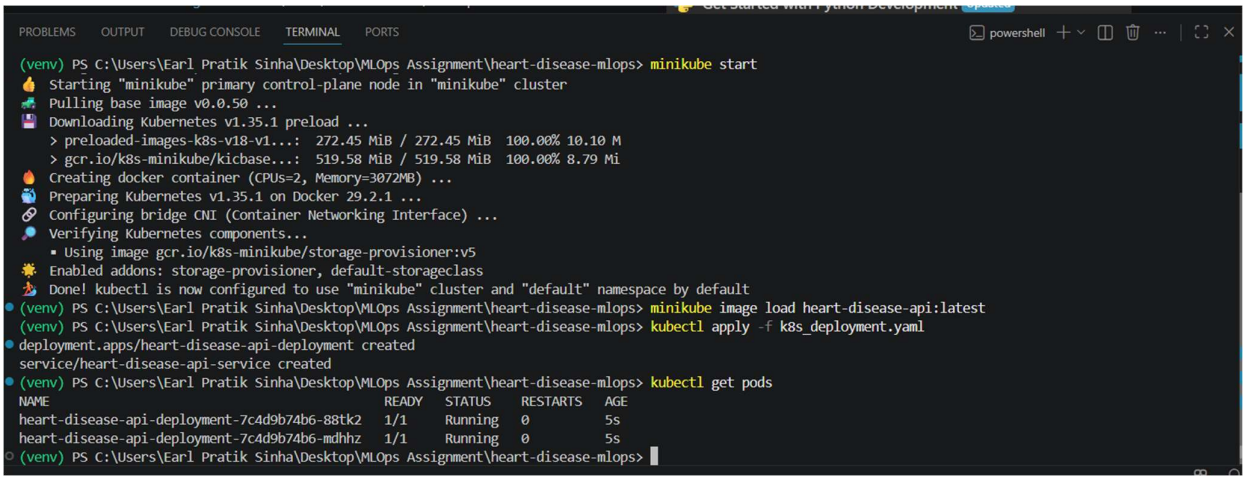
Public GitHub Repository: https://github.com/earlpratiksinha/heart-disease-mlops

4.1. CI/CD Pipeline Success

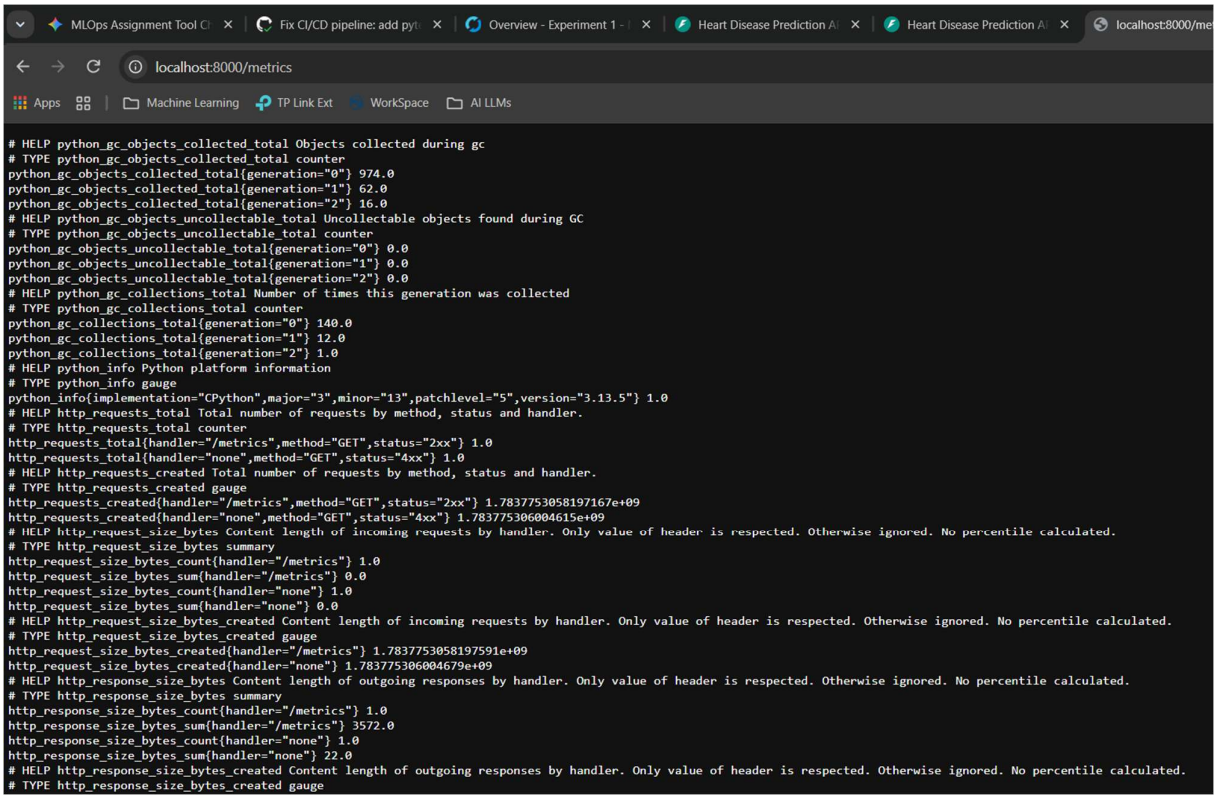


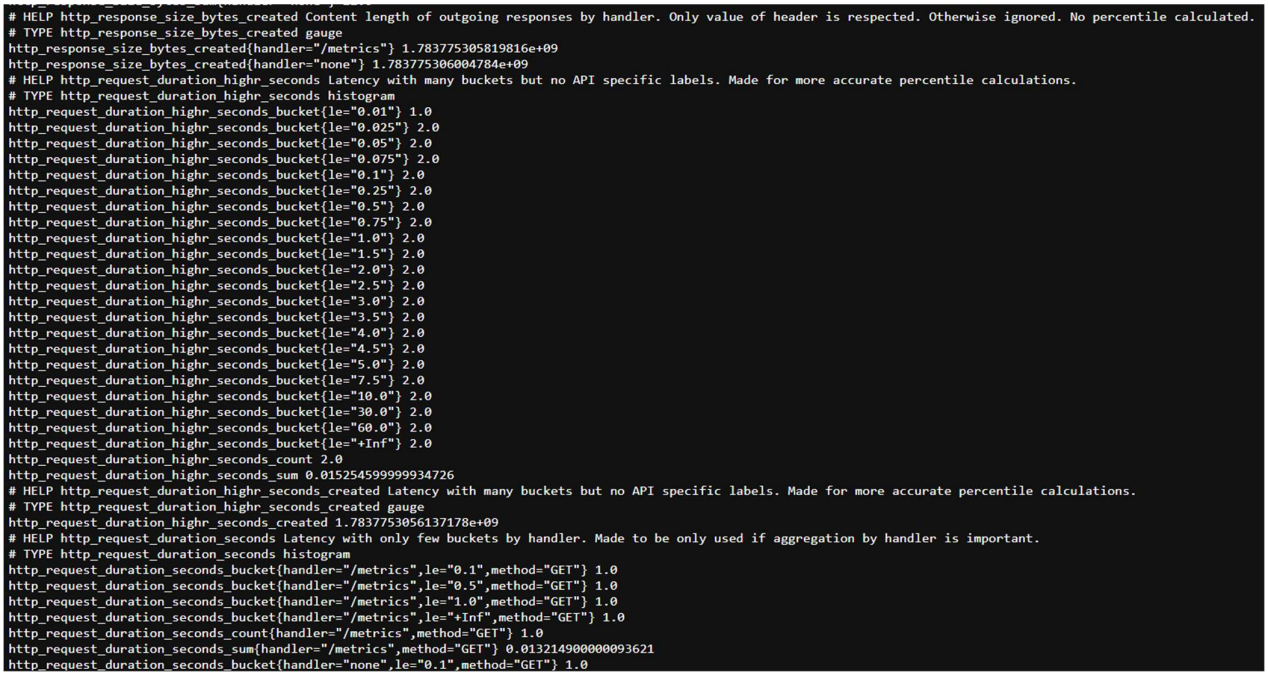


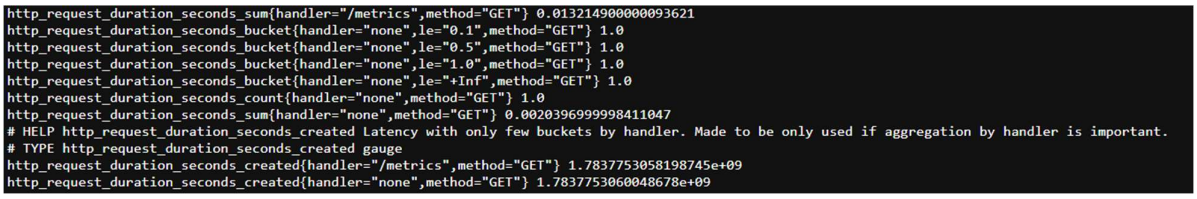
4.2. Kubernetes Deployment



4.3. Prometheus Monitoring







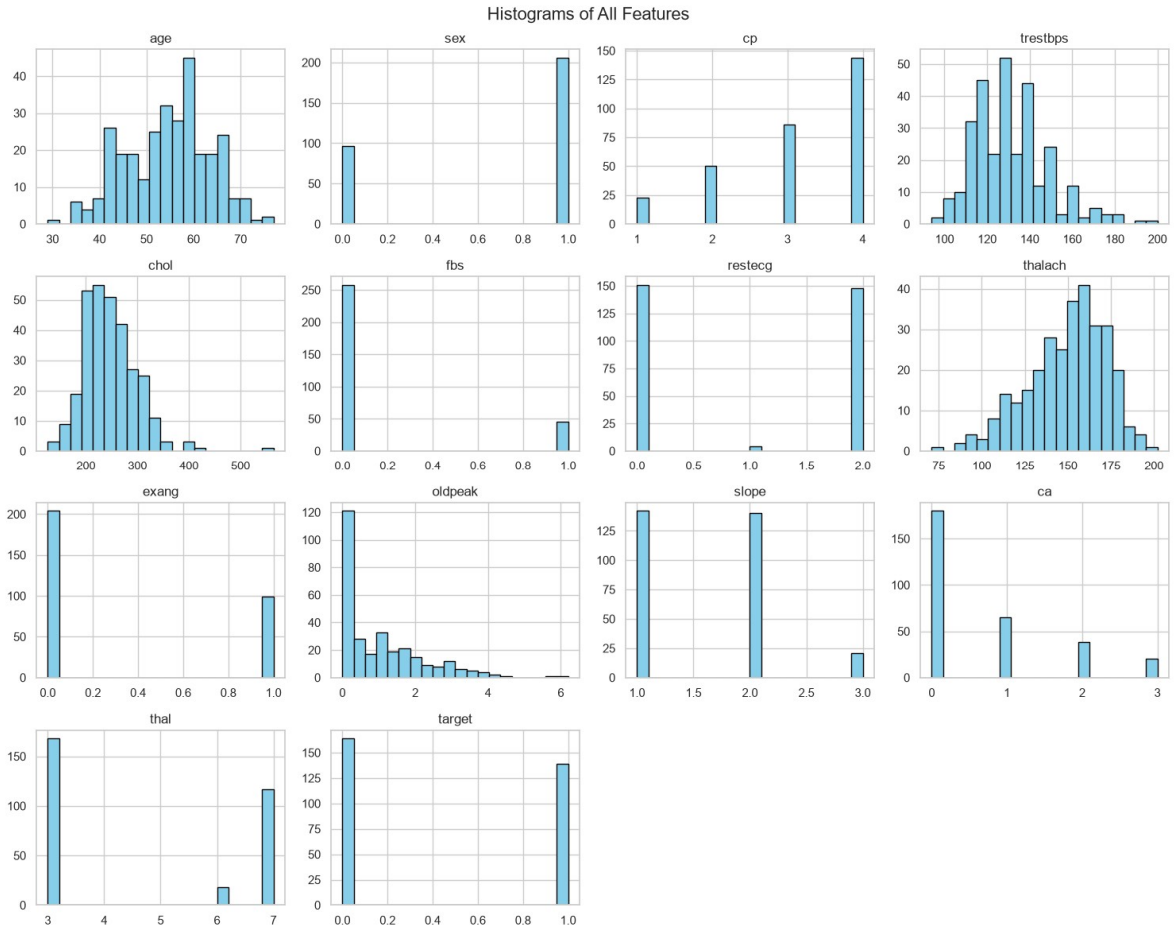
Link to code repository - https://github.com/earlpratiksinha/heart-disease-mlops

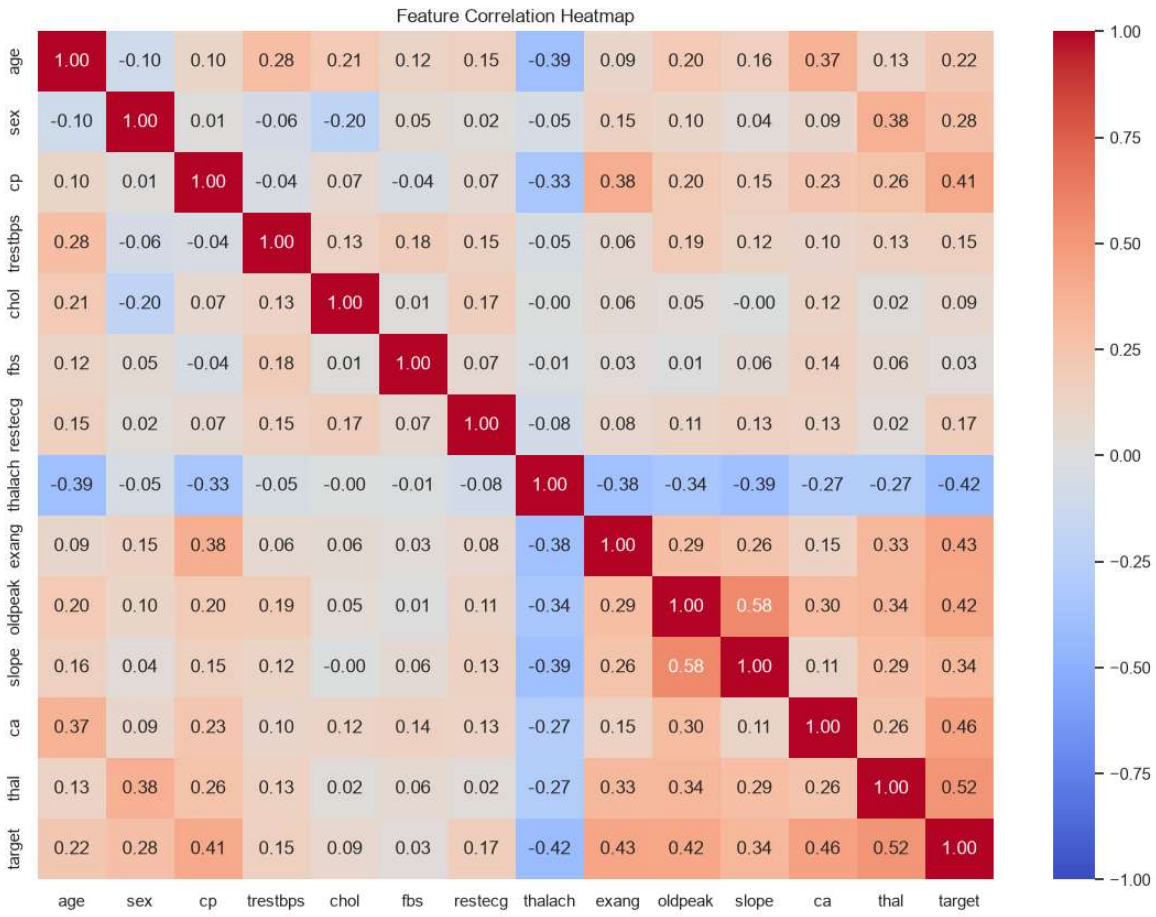
5. EDA and Modelling Choices

Exploratory Data Analysis (EDA) & Data Cleaning

The dataset used is the UCI Heart Disease dataset. During the EDA and cleaning phase (documented in notebooks/01_eda_and_cleaning.ipynb and automated in src/download_data.py), the following steps were taken:

- **Target Variable Transformation:** The original target variable contained multiple classes representing the severity of heart disease. To frame this as a standard binary classification problem, the target was binarized (0 = no disease, 1 = presence of disease).
- **Handling Missing Values:** Missing values were identified in the dataset. To prevent data loss and maintain the distribution, missing continuous values were imputed using the median of their respective columns.


Histograms of All Features
age
sex
cp
trestbps
chol
fbs
restecg
thalach
exang
oldpeak
slope
ca
thal
target



Modelling Choices

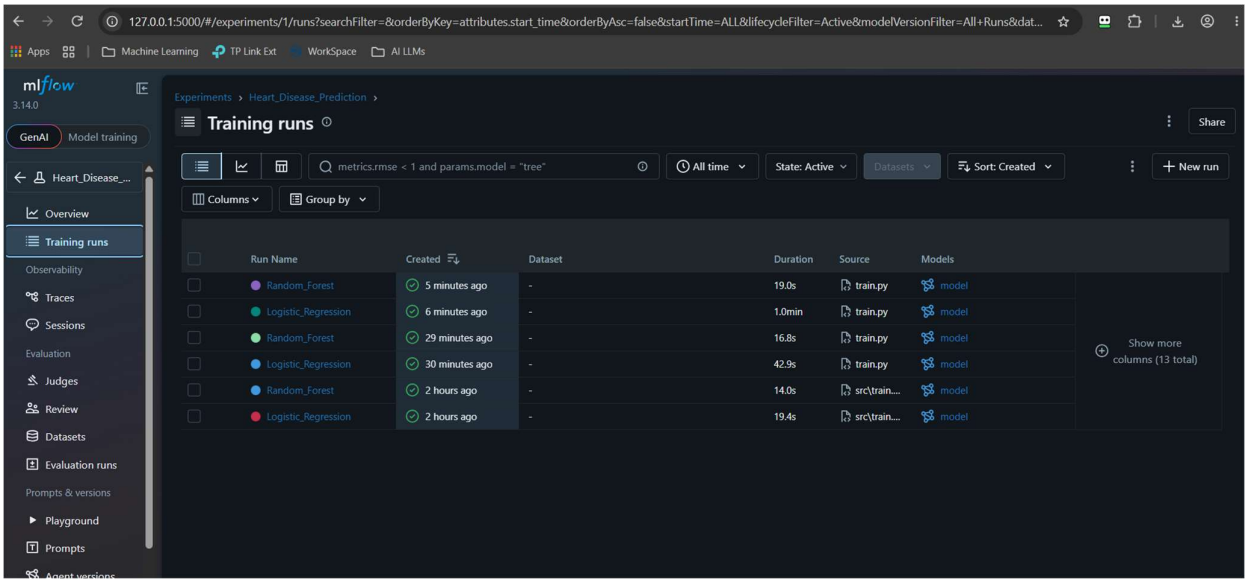
- **Feature Selection:** All available physiological features (e.g., age, cholesterol, resting blood pressure, max heart rate) were utilized as they are clinically relevant to heart disease prediction.
- **Model Selection:** An automated champion/challenger approach was implemented. Both a Logistic Regression model (to establish a highly interpretable baseline) and a Random Forest Classifier (to capture complex, non-linear relationships) were trained within a pipeline containing a StandardScaler. The system automatically compared their performance using the ROC-AUC metric via MLflow, and dynamically selected the best-performing model to be serialized for the production API.Evaluation: The model's performance was evaluated using standard classification metrics, ensuring a balance between precision and recall, which is critical in healthcare diagnostics.

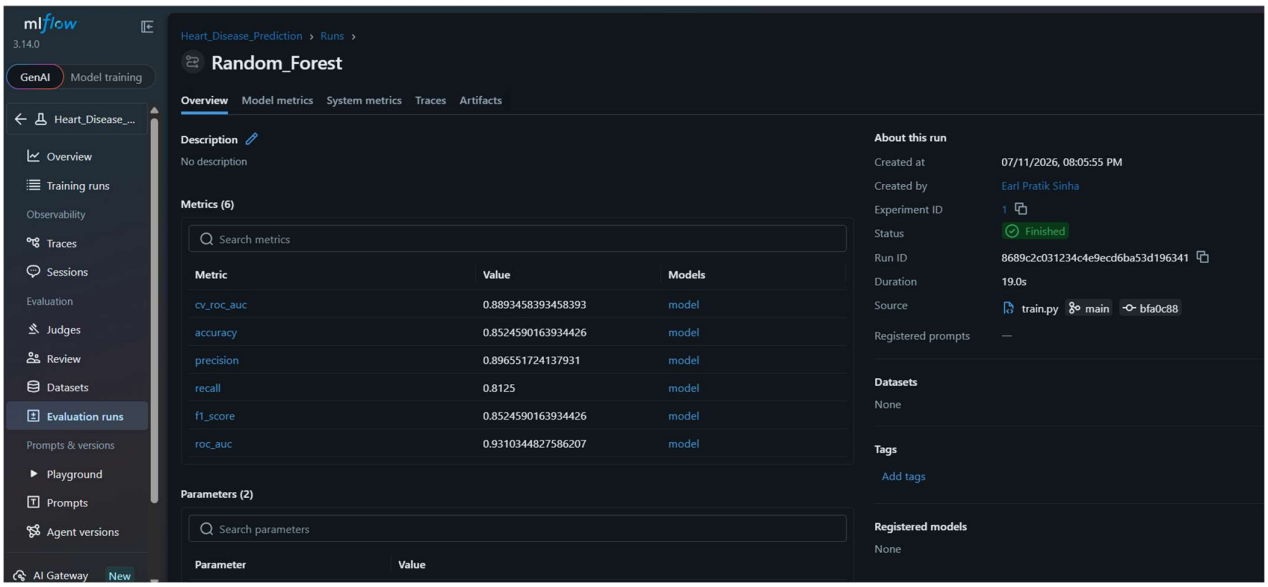
6. Experiment Tracking Summary

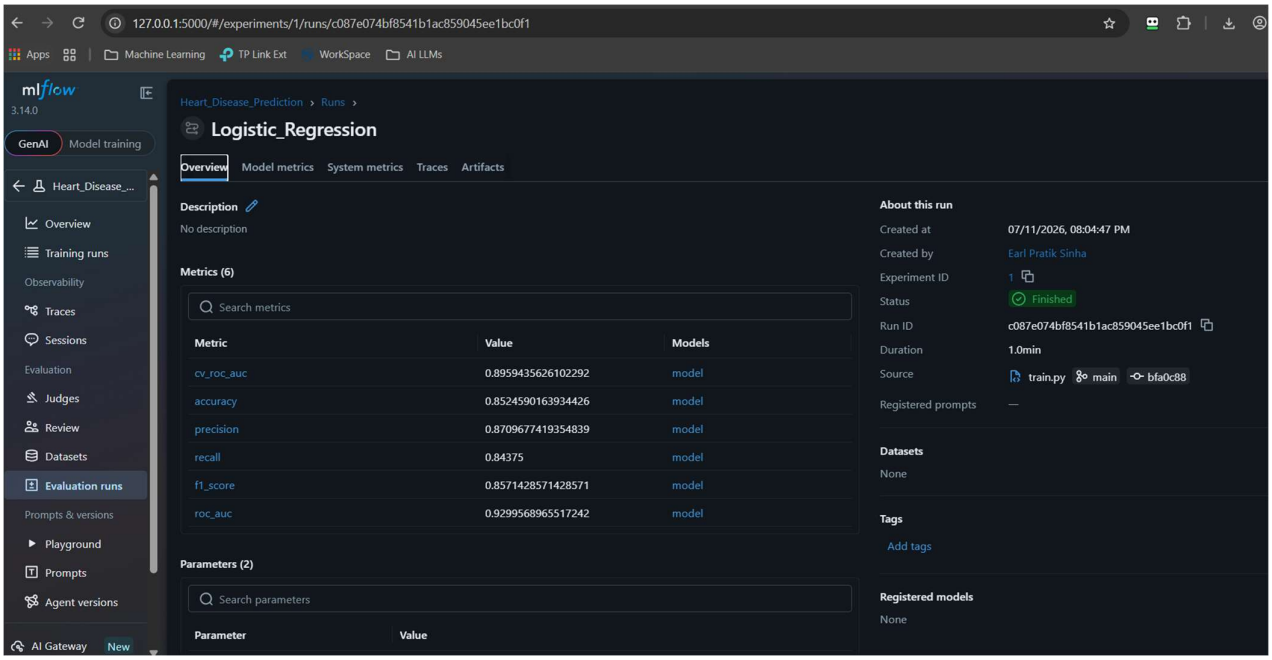
Model training runs and hyperparameters were tracked using **MLflow**.

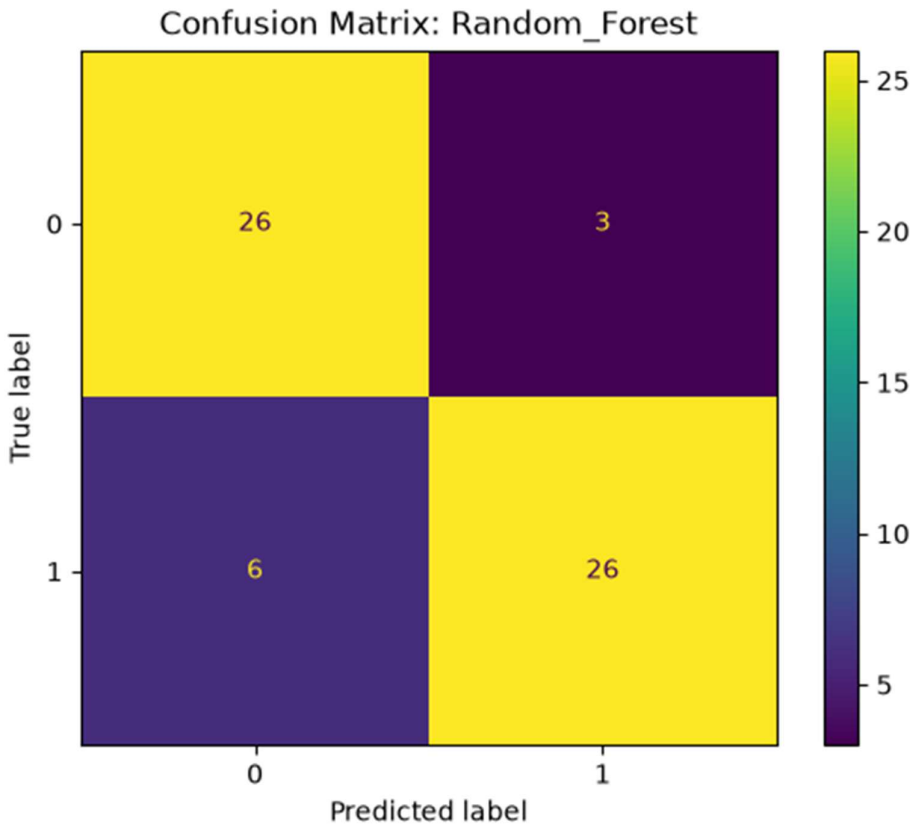
- A local SQLite database (mlflow.db) was utilized as the tracking backend to log experiment data.

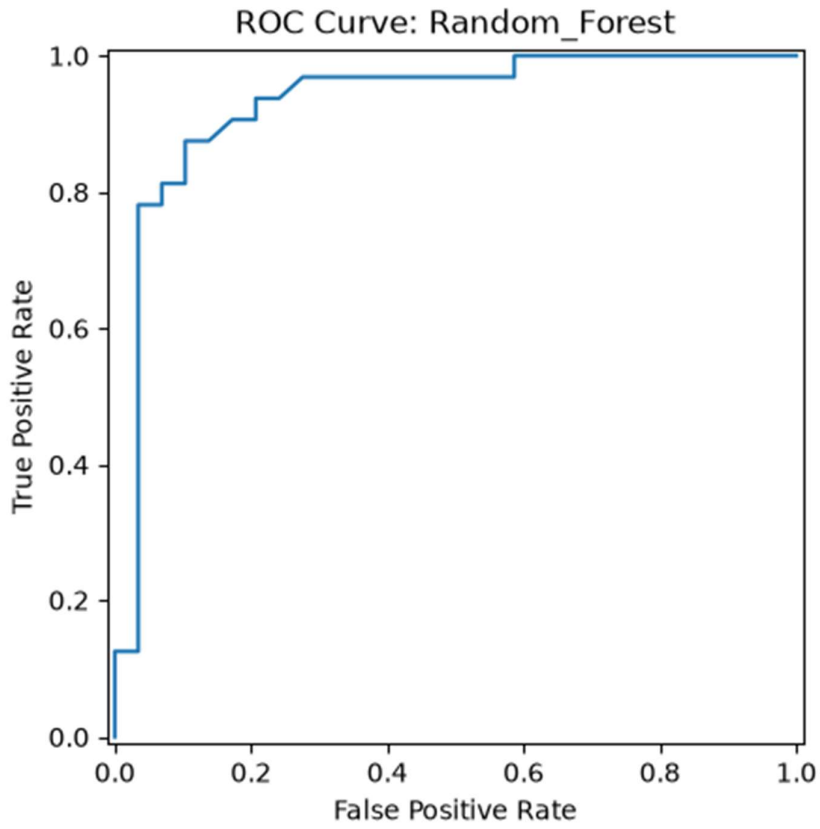
- MLflow tracked experiment parameters, evaluation metrics (Accuracy, Precision, Recall, F1-score and ROC-AUC), the trained model, and artifacts including ROC Curve and Confusion Matrix visualizations.
- The system automatically identifies the best-performing model from these tracked runs and serializes it as models/best_model.pkl to be served by the FastAPI application.

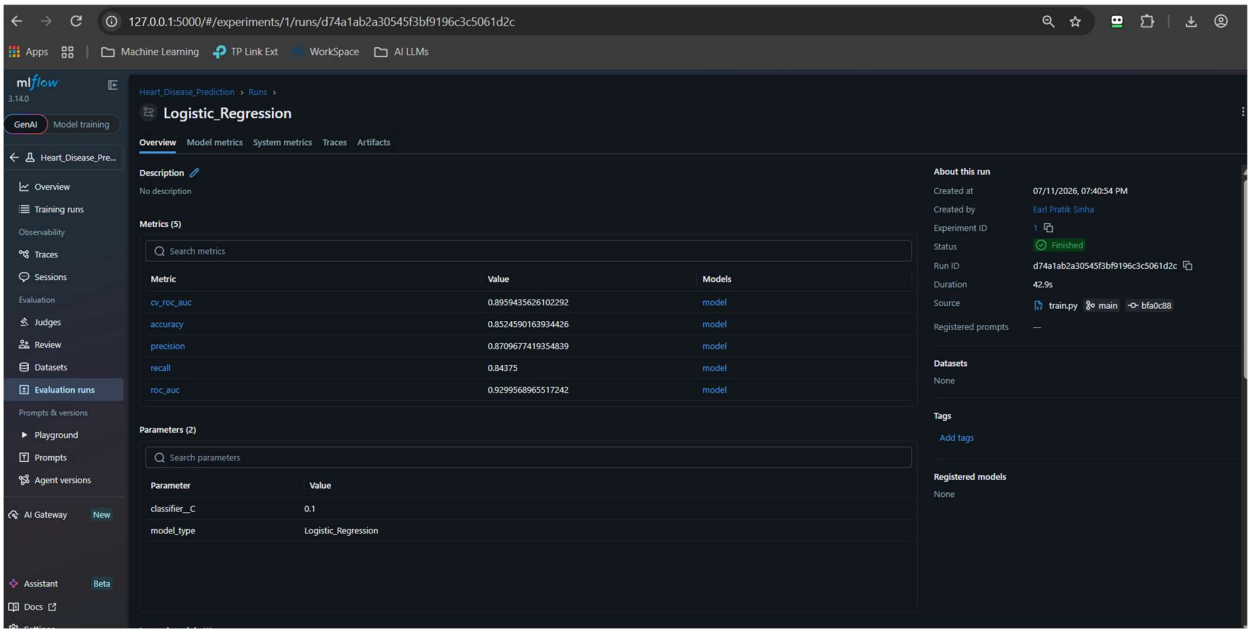





127.0.0.1:5000/#/experiments/1/runs/c087e074bf8541b1ac859045ee1bc0f1
Apps
Machine Learning
TP Link Ext
WorkSpace
AI LLMs
mlflow
3.14.0
GenAI
Model training
Heart_Disease_...
Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Review
Datasets
Evaluation runs
Prompts & versions
Playground
Prompts
Agent versions
AI Gateway
New
Heart_Disease_Prediction › Runs ›
Logistic_Regression
Overview
Model metrics
System metrics
Traces
Artifacts
Description
No description
Metrics (6)
Search metrics
Metric
Value
Models
cv_roc_auc
0.8959435626102292
model
accuracy
0.8524590163934426
model
precision
0.8709677419354839
model
recall
0.84375
model
f1_score
0.8571428571428571
model
roc_auc
0.9299568965517242
model
Parameters (2)
Search parameters
Parameter
Value
About this run
Created at
07/11/2026, 08:04:47 PM
Created by
Earl Pratik Sinha
Experiment ID
1
Status
Finished
Run ID
c087e074bf8541b1ac859045ee1bc0f1
Duration
1.0min
Source
train.py
main
bfa0c88
Registered prompts
—
Datasets
None
Tags
Add tags
Registered models
None

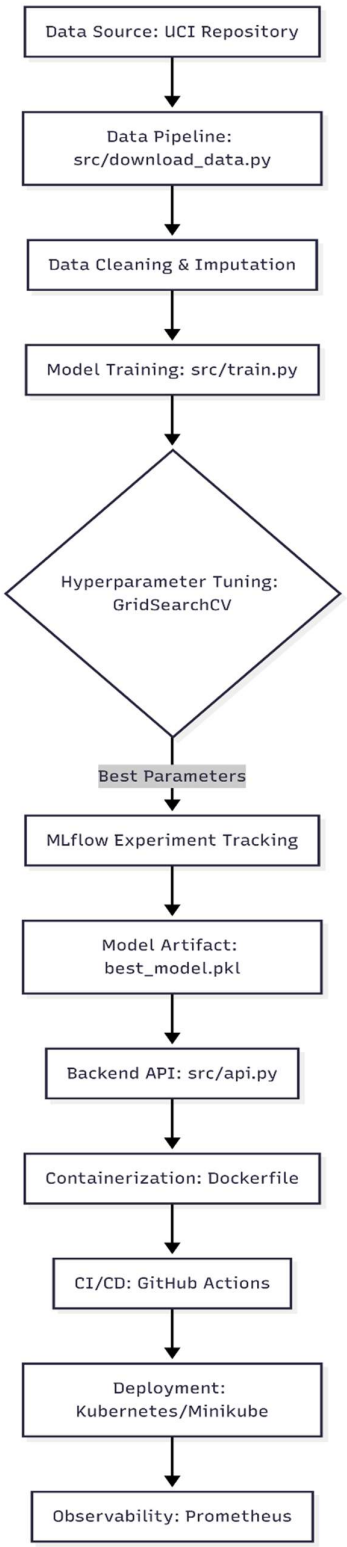

Confusion Matrix: Random_Forest
True label
Predicted label
0
1
26
3
6
26
25
20
15
10
5


ROC Curve: Random_Forest
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0

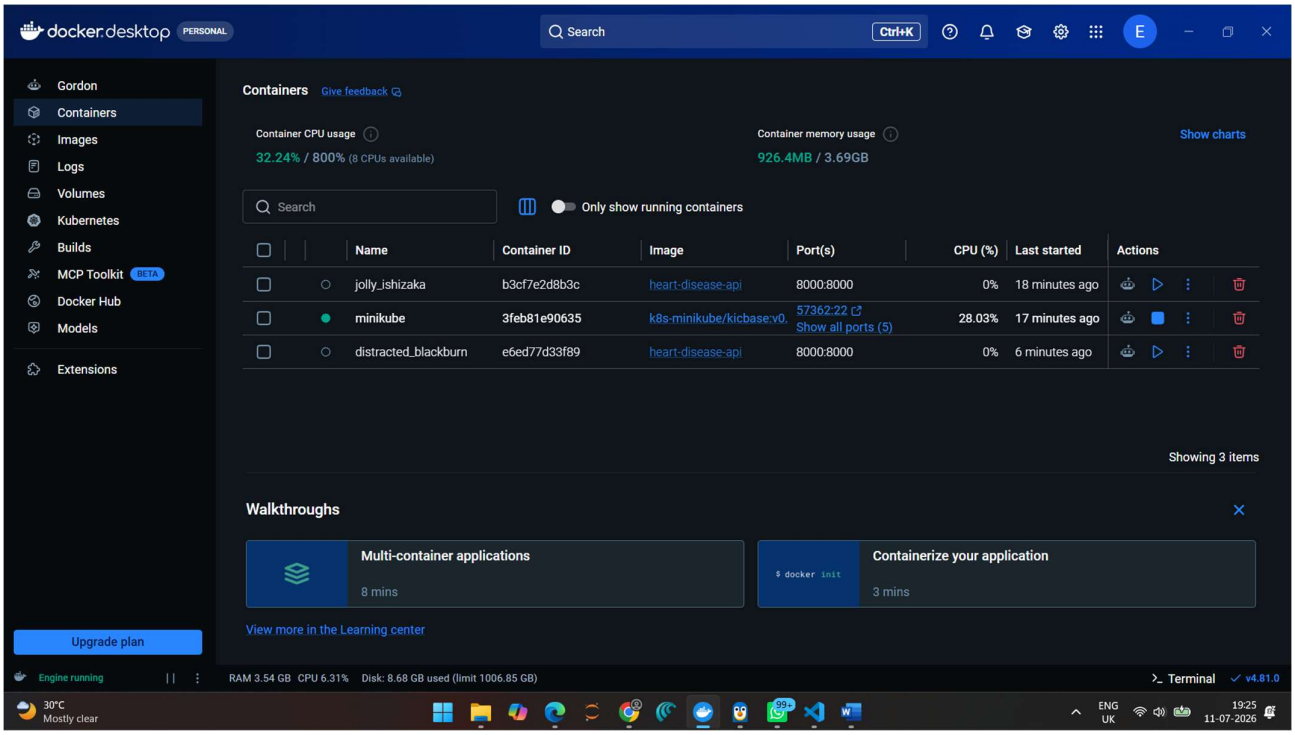

127.0.0.1:5000/#/experiments/1/runs/d74a1ab2a30545f3bf9196c3c5061d2c
Apps
Machine Learning
TP Link Ext
WorkSpace
AI LLMs
mlflow
3.14.0
GenAI
Model training
Heart_Disease_Pre...
Overview
Training runs
Observability
Traces
Sessions
Evaluation
Judges
Review
Datasets
Evaluation runs
Prompts & versions
Playground
Prompts
Agent versions
AI Gateway
New
Assistant
Beta
Docs
Heart_Disease_Prediction › Runs ›
Logistic_Regression
Overview
Model metrics
System metrics
Traces
Artifacts
Description
No description
Metrics (5)
Search metrics
Metric
Value
Models
cv_roc_auc
0.8959435626102292
model
accuracy
0.8524590163934426
model
precision
0.8709677419354839
model
recall
0.84375
model
roc_auc
0.929956896551724
model
Parameters (2)
Search parameters
Parameter
Value
classifier__C
0.1
model_type
Logistic_Regression
About this run
Created at
07/11/2026, 07:40:54 PM
Created by
Earl Pratik Sinha
Experiment ID
1
Status
Finished
Run ID
d74a1ab2a30545f3bf9196c3c5061d2c
Duration
42.9s
Source
train.py
main
bfa0c88
Registered prompts
—
Datasets
None
Tags
Add tags
Registered models
None

7. Architecture Diagram

Below is the high-level architecture of the MLOps pipeline:



Docker –



Video link - https://drive.google.com/file/d/1H8ercOb-Ap6BFAoywdTbK4sRwrROROdS/view?usp=sharing

Git - https://github.com/earlpratiksinha/heart-disease-mlops